

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра ВТ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Основы машинного обучения»
Тема: «Классификация»

Студент гр. 2308

Кислицин К. Р.

Преподаватель

Турсуков Н. О.

Санкт-Петербург

2024

Содержание

Цель работы	3
Задачи работы	3
Теоретическая информация	3
Ход работы.....	6
Выводы	19
Приложения	20

Цель работы

Целью работы является получение и закрепление навыков предобработки данных и применения методов машинного обучения для решения задач классификации.

Задачи работы

1. Предварительная обработка данных

- Визуализация значимых признаков (диаграммы рассеяния, ящики с усами, гистограммы)
- Очистка данных (удаление пропусков, нормализация, удаление дубликатов)
- Корреляция данных (матрица корреляций)

2. Обучение моделей и подбор параметров:

- К-ближайших соседей (KNN)
- Машина опорных векторов (SVM)
- Дерево решений ИЛИ Случайный лес

3. Оценка моделей

- Визуализация предсказанных значений
- Оценка качества прогноза (precision/recall/f1-score/ROC-AUC)
- Визуализация дерева решений ИЛИ Визуализация Feature Importance для случайного леса

Теоретическая информация

Визуализация значимых признаков

- Диаграммы рассеяния (scatter plots): визуализируют зависимость между двумя количественными признаками, что помогает выявить линейные и нелинейные взаимосвязи, а также обнаружить выбросы.
- Ящики с усами (boxplots): позволяют оценить распределение признаков, медиану, квартильные значения и возможные выбросы в данных.

- Гистограммы (histograms): отображают распределение данных по категориям или интервалам для одного признака, помогают оценить симметрию, наличие выбросов и форму распределения.

Матрица корреляций – квадратная таблица, заголовками строк и столбцов которой являются обрабатываемые переменные, а на пересечении строк и столбцов выводятся коэффициенты корреляции для соответствующей пары признаков.

К-ближайших соседей (KNN)

- KNN – это алгоритм классификации, основанный на нахождении ближайших объектов в пространстве признаков.
- Основная идея: для классификации нового объекта алгоритм находит K ближайших объектов в тренировочной выборке и определяет класс нового объекта на основе большинства среди этих K соседей.
- Подбор параметров: Основной параметр – это число соседей K. Также важен выбор метрики расстояния (например, Евклидово, Манхэттена).

Машина опорных векторов (SVM)

- SVM – это алгоритм, который строит гиперплоскость или гиперповерхность для разделения данных на классы.
- Основная цель: найти такую гиперплоскость, которая максимально увеличивает отступ (margin) между классами, используя только те объекты, которые находятся на границах классов (опорные векторы).
- Подбор параметров: Важные параметры включают регуляризацию C, выбор ядра (линейное, полиномиальное, гауссово) и параметры ядра.

Случайный лес

- Случайный лес – это ансамблевый метод, который строит множество деревьев решений и усредняет их предсказания для улучшения

устойчивости и точности. Дерево решений – это алгоритм, который строит древовидную структуру для принятия решений, где каждый узел представляет собой проверку условия по одному из признаков, а листья представляют классы.

- Основная идея: случайное подмножество признаков и данных для каждого дерева позволяет избежать переобучения и повысить обобщающую способность модели.
- Подбор параметров: Количество деревьев, глубина деревьев, количество признаков, выбранных на каждой итерации.

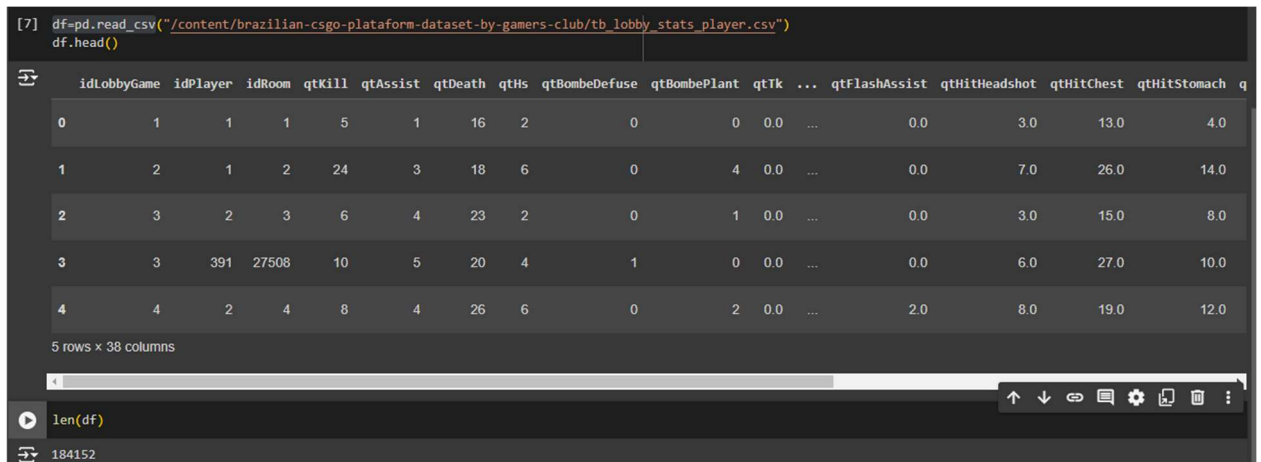
Оценка качества прогноза

- Precision (точность): отношение числа правильно предсказанных объектов данного класса ко всем объектам, предсказанным как данный класс.
- Recall (полнота): отношение числа правильно предсказанных объектов данного класса ко всем объектам истинного класса.
- F1-score: гармоническое среднее между Precision и Recall, используется для оценки баланса между этими метриками.
- ROC-AUC (Area Under Curve): площадь под кривой ROC, показывает соотношение между чувствительностью и специфичностью для различных порогов классификации.

Ход работы

В рамках данной лабораторной работы был рассмотрен датасет по статистике игроков бразильского геймерского клуба в CS:GO. Датасет содержит в себе 184152 строки и 38 признаков.

Сформировал датафрейм на основе загруженных данных и вывел его первые 5 значений (Рисунок 1).



```
[7] df=pd.read_csv("/content/brazilian-csgo-plataform-dataset-by-gamers-club/tb_lobby_stats_player.csv")
df.head()
```

	idLobbyGame	idPlayer	idRoom	qtKill	qtAssist	qtDeath	qtHs	qtBombeDefuse	qtBombePlant	qtTk	...	qtFlashAssist	qtHitHeadshot	qtHitChest	qtHitStomach	q
0	1	1	1	5	1	16	2	0	0	0.0	...	0.0	3.0	13.0	4.0	
1	2	1	2	24	3	18	6	0	4	0.0	...	0.0	7.0	26.0	14.0	
2	3	2	3	6	4	23	2	0	1	0.0	...	0.0	3.0	15.0	8.0	
3	3	391	27508	10	5	20	4	1	0	0.0	...	0.0	6.0	27.0	10.0	
4	4	2	4	8	4	26	6	0	2	0.0	...	2.0	8.0	19.0	12.0	

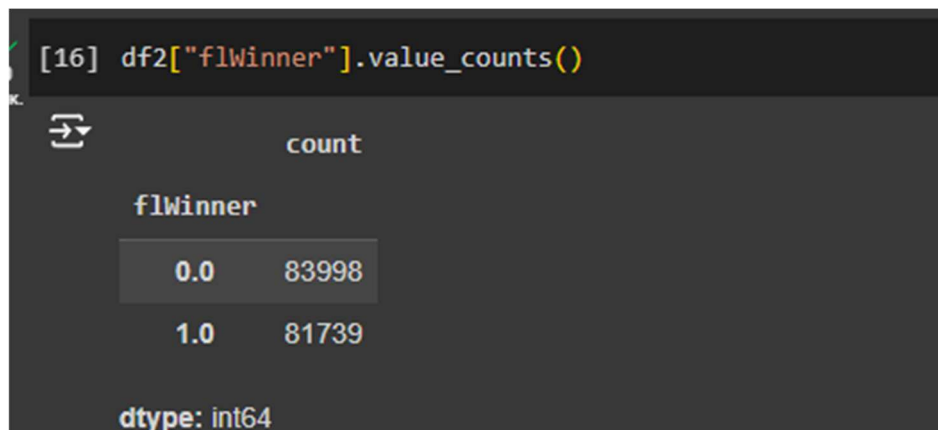
5 rows x 38 columns

```
len(df)
```

184152

Рисунок 1 – Предосмотр датафрейма

В качестве цели предсказания взял общий исход игры, т.е. классов получилось два. Посмотрел количество строк каждого класса и заключил, что они сбалансированы (Рисунок 2)



```
[16] df2["flWinner"].value_counts()
```

flWinner	count
0.0	83998
1.0	81739

dtype: int64

Рисунок 2 – Баланс классов

Построил карту пропусков, пропусков не оказалось, потому добавил их искусственно в два столбца. Соответствующие действия показаны на Рисунке 3.



Рисунок 3 – Добавление пропусков

Посмотрел тип данных в столбцах для того, чтобы определить дальнейшие действия по обработке. Нашлось два признака с типом данных «object»: название карты и дата создания записи. Дату разбил на несколько столбцов, название – закодировал. Убрал строки, в которых есть пустые значения для предсказываемого параметра. Все действия приведены на рисунках 4-5.

```
df2.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 184152 entries, 0 to 184151
Data columns (total 38 columns):
#   Column                Non-Null Count  Dtype
---  -
0   idLobbyGame            184152 non-null int64
1   idPlayer               184152 non-null int64
2   idRoom                 184152 non-null int64
3   qtKill                 184152 non-null int64
4   qtAssist               184152 non-null int64
5   qtDeath                184152 non-null int64
6   qtHs                   184152 non-null int64
7   qtBombeDefuse          184152 non-null int64
8   qtBombePlant           184152 non-null int64
9   qtTk                   184032 non-null float64
10  qtTkAssist             184032 non-null float64
11  qt1Kill                184152 non-null int64
12  qt2Kill                184152 non-null int64
13  qt3Kill                184152 non-null int64
14  qt4Kill                184152 non-null int64
15  qt5Kill                184152 non-null int64
16  qtPlusKill             184152 non-null int64
17  qtFirstKill            184152 non-null int64
18  vlDamage               184152 non-null int64
19  qtHits                 184032 non-null float64
20  qtShots                184152 non-null int64
21  qtLastAlive            184032 non-null float64
22  qtClutchWon            184152 non-null int64
23  qtRoundsPlayed         184152 non-null int64
24  descMapName            184152 non-null object
25  vlLevel                184152 non-null int64
26  qtSurvived             183447 non-null float64
27  qtTrade                183447 non-null float64
28  qtFlashAssist          183447 non-null float64
29  qtHitHeadshot          183447 non-null float64
30  qtHitChest             183447 non-null float64
31  qtHitStomach           165089 non-null float64
32  qtHitLeftArm           183447 non-null float64
33  qtHitRightArm          183447 non-null float64
34  qtHitLeftLeg           183447 non-null float64
35  qtHitRightLeg          183447 non-null float64
36  flWinner               165737 non-null float64
37  dtCreatedAt            184152 non-null object
dtypes: float64(15), int64(21), object(2)
memory usage: 53.4+ MB
```

Рисунок 4 – Вывод типов данных

```
[18] df2 = df2.dropna(subset=['flWinner'])

[21] df2['dtCreatedAt'] = pd.to_datetime(df2['dtCreatedAt'])

df2['year'] = df2['dtCreatedAt'].dt.year
df2['month'] = df2['dtCreatedAt'].dt.month
df2['day'] = df2['dtCreatedAt'].dt.day
df2['dayofweek'] = df2['dtCreatedAt'].dt.dayofweek
df2['hour'] = df2['dtCreatedAt'].dt.hour
df2['quarter'] = df2['dtCreatedAt'].dt.quarter

df2["descMapName"].value_counts()

count
descMapName
de_mirage    52293
de_inferno   37875
de_dust2     21391
de_vertigo   17956
de_overpass  12738
de_nuke      12841
de_train     7033
de_ancient   3810

dtype: int64

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

df2['descMapName'] = le.fit_transform(df2['descMapName'])
df2["descMapName"].value_counts()

Показать скрытые выходные данные

[27] df2.info()

Показать скрытые выходные данные

[26] df2=df2.drop(["dtCreatedAt"], axis=1)
```

Рисунок 5 – Кодирование данных

Построил диаграммы рассеивания для 10 случайных параметров (т.к. количество параметров велико), чтобы посмотреть, как распределены классы (Рисунок 6).

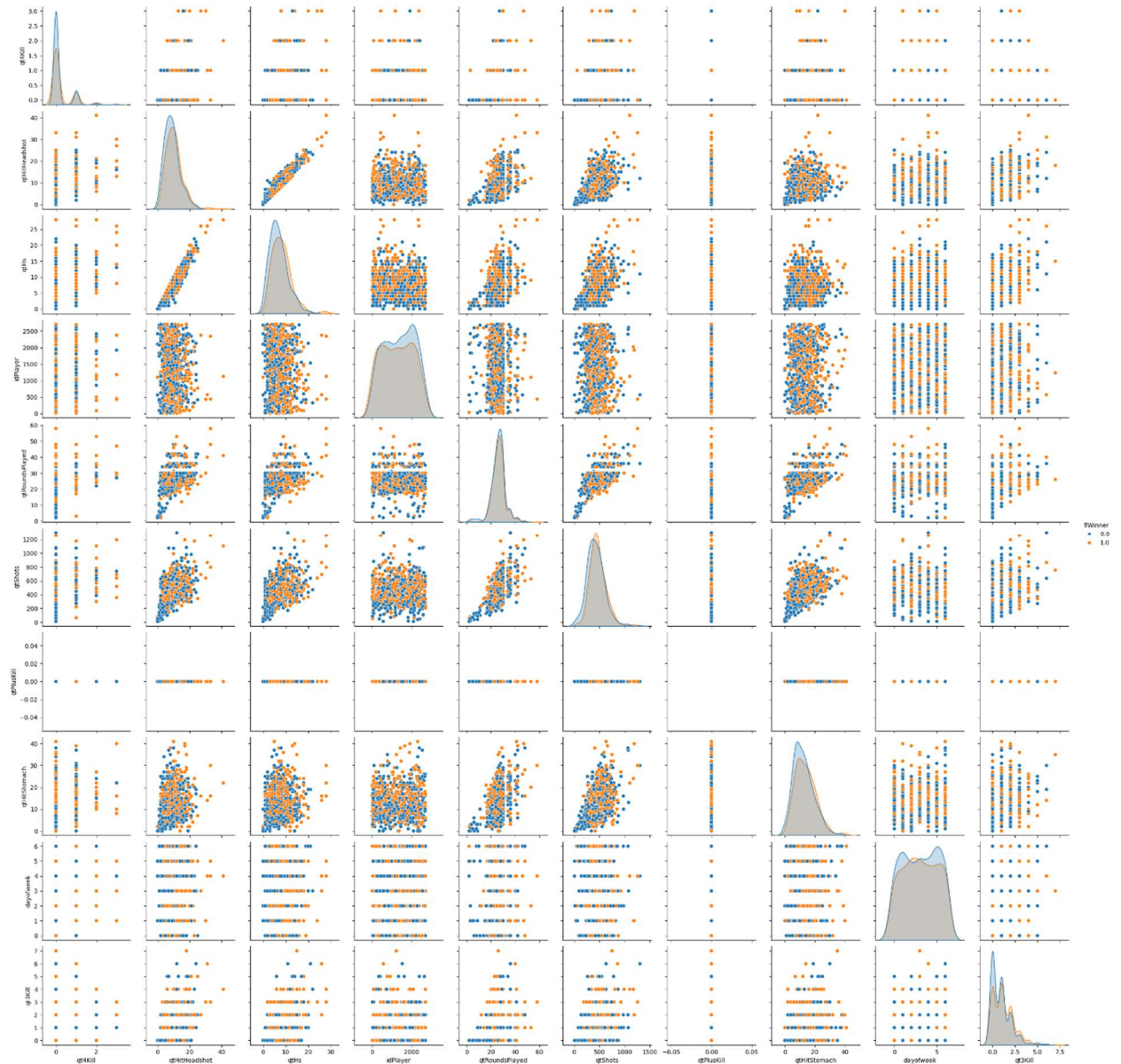


Рисунок 6 – Диаграммы рассеивания для 10 параметров

По построенным диаграммам можно сказать, что у классов четкие границы отсутствуют, что предполагает сложность классификации. Само распределение классов является нормальным для каждого оцененного параметра.

Дальше построил матрицу корреляции данных (Рисунок 7).

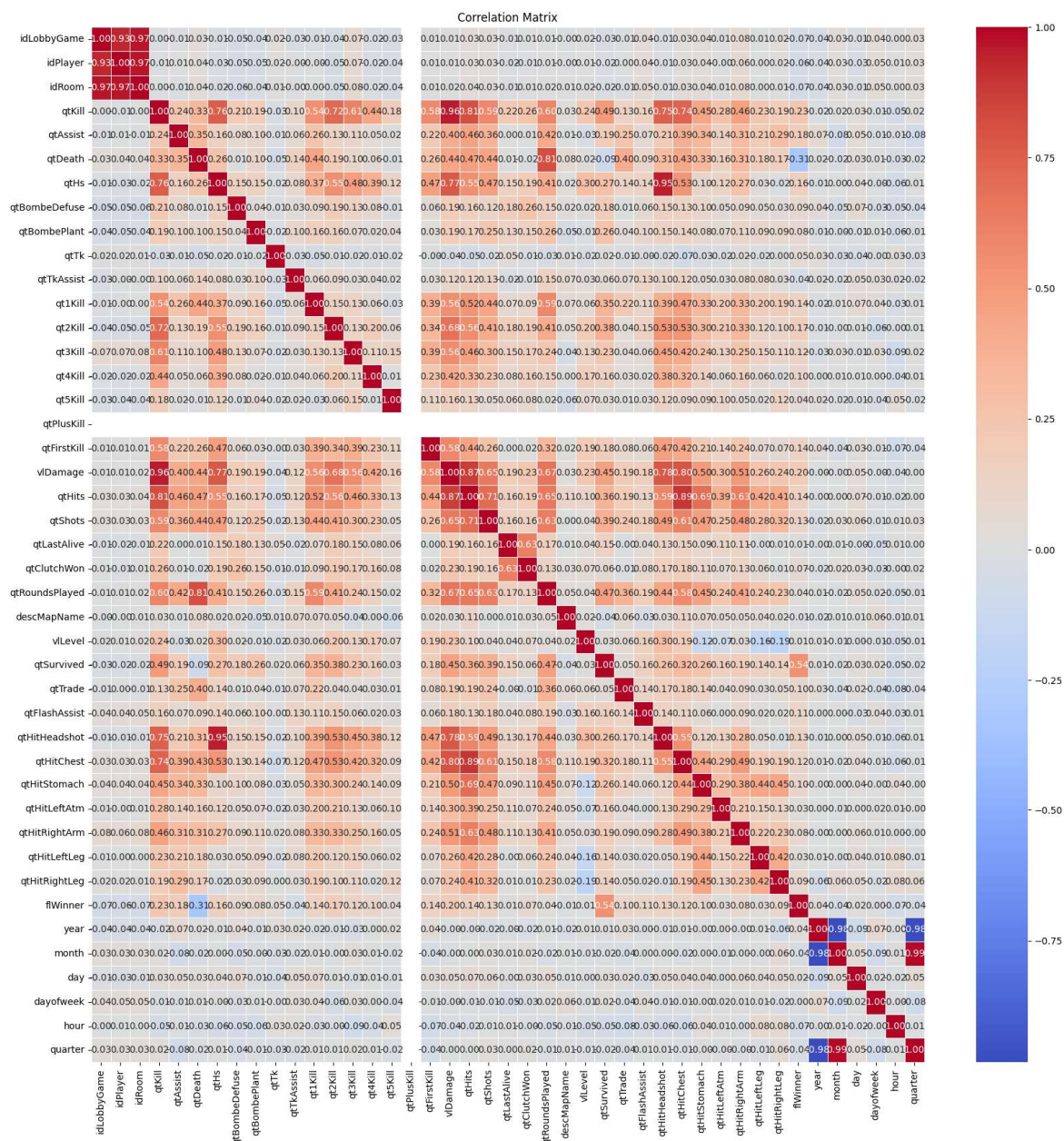


Рисунок 7 – Матрица корреляции данных

По данной матрице были определены параметры, наименее влияющие на другие, у которых коэффициент близок к 0. Удалил эти параметры и заполнил пропуски медианным значением (Рисунок 8).

```
df3 = df3.fillna(df3.median())

[49] df3 = df3.drop(['qtTk', 'day', 'year', 'month', 'qtPlusKill'], axis=1)
```

Рисунок 8 – Заполнение пропусков в столбцах

Построил ящики с усами для определения выбросов в данных (Рисунок 9).

Дальше выполнил нормализацию признаков и подготовил данные для обучения (Рисунок 11).

```
[58] from sklearn.preprocessing import MinMaxScaler
      scaler = MinMaxScaler()
      df3 = pd.DataFrame(scaler.fit_transform(df3), columns=df3.columns)

[59] x = df3.drop(['flWinner'], axis=1)
      y = df3['flWinner']
```

Рисунок 11 – Завершение предобработки данных

Так как количество данных велико, обучение можно провести на меньшей выборке, с возможностью увеличить её в случае плохих результатов. Выборка представлена на Рисунке 12.

```
[60] x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)
```

Рисунок 12 – Выборка данных для обучения

Обучил модель KNN и предсказал значения на тестовой выборке (Рисунок 13).

```
▶ knn = KNeighborsClassifier(n_neighbors=5)
  knn.fit(x_train, y_train)

↔ KNeighborsClassifier ⓘ ⓘ
  KNeighborsClassifier()

[68] y_pred = knn.predict(x_test)
      y_pred_proba = knn.predict_proba(x_test)[:, 1]
```

Рисунок 13 – Обучение KNN

Вывел метрики качества модели (Рисунок 14). Построил матрицу ошибок (Рисунок 15). Построил ROC-кривые (Рисунок 16).

```

Accuracy: 0.64
ROC AUC Score: 0.69

Classification Report:

```

	precision	recall	f1-score	support
0.0	0.64	0.69	0.66	1695
1.0	0.65	0.58	0.61	1618
accuracy			0.64	3313
macro avg	0.64	0.64	0.64	3313
weighted avg	0.64	0.64	0.64	3313

Рисунок 14 – Метрики качества KNN

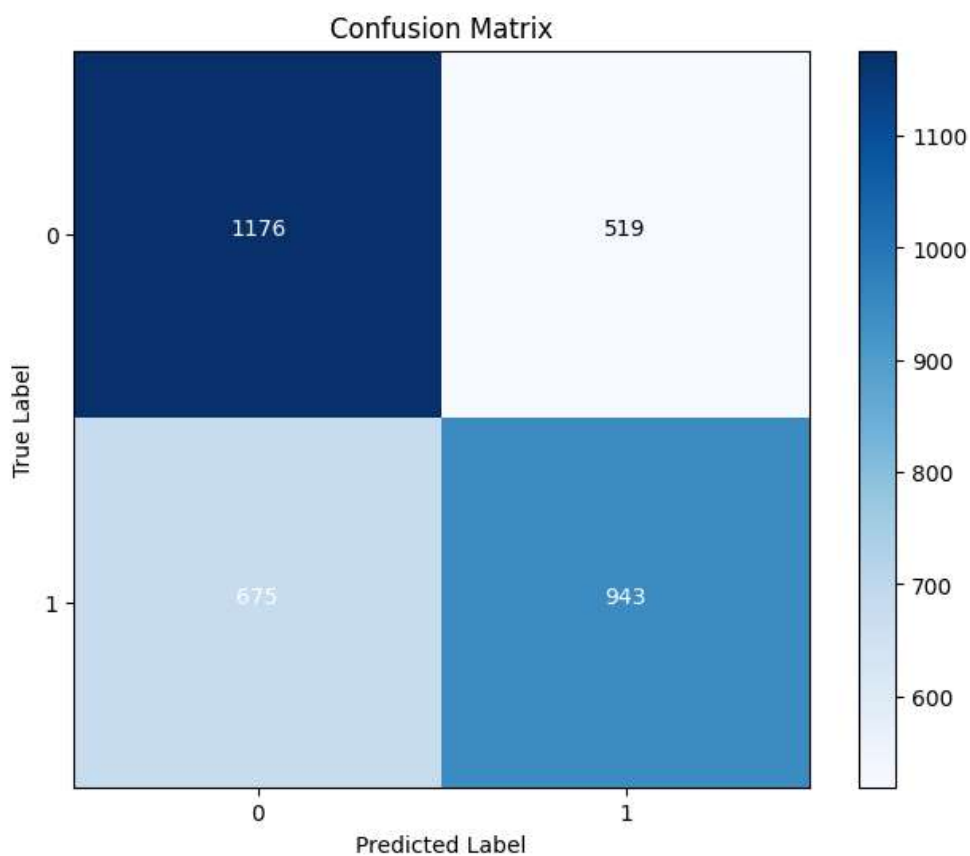


Рисунок 15 – Матрица ошибок KNN

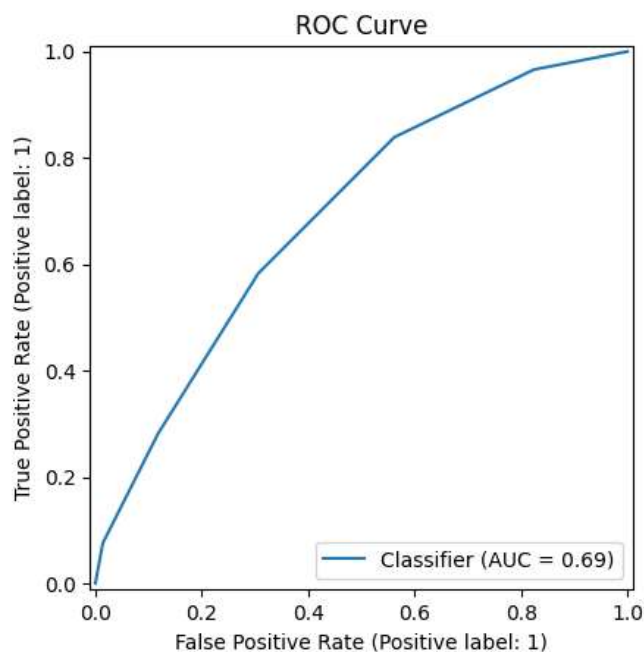


Рисунок 16 – ROC-кривые для KNN

Обучил модель SVM и предсказал значения на тестовой выборке (Рисунок 17).

```
from sklearn.svm import SVC
svm = SVC(probability=True)
svm.fit(x_train, y_train)

SVC
SVC(probability=True)

[74] y_pred = svm.predict(x_test)
     y_pred_proba = svm.predict_proba(x_test)[: , 1]
```

Рисунок 17 – Обучение SVM

Вывел метрики качества модели (Рисунок 18). Построил матрицу ошибок (Рисунок 19). Построил ROC-кривые (Рисунок 20).

Accuracy: 0.80
ROC AUC Score: 0.88

Classification Report:

	precision	recall	f1-score	support
0.0	0.80	0.81	0.80	1695
1.0	0.79	0.78	0.79	1618
accuracy			0.80	3313
macro avg	0.80	0.79	0.79	3313
weighted avg	0.80	0.80	0.80	3313

Рисунок 18 – Метрики качества SVM

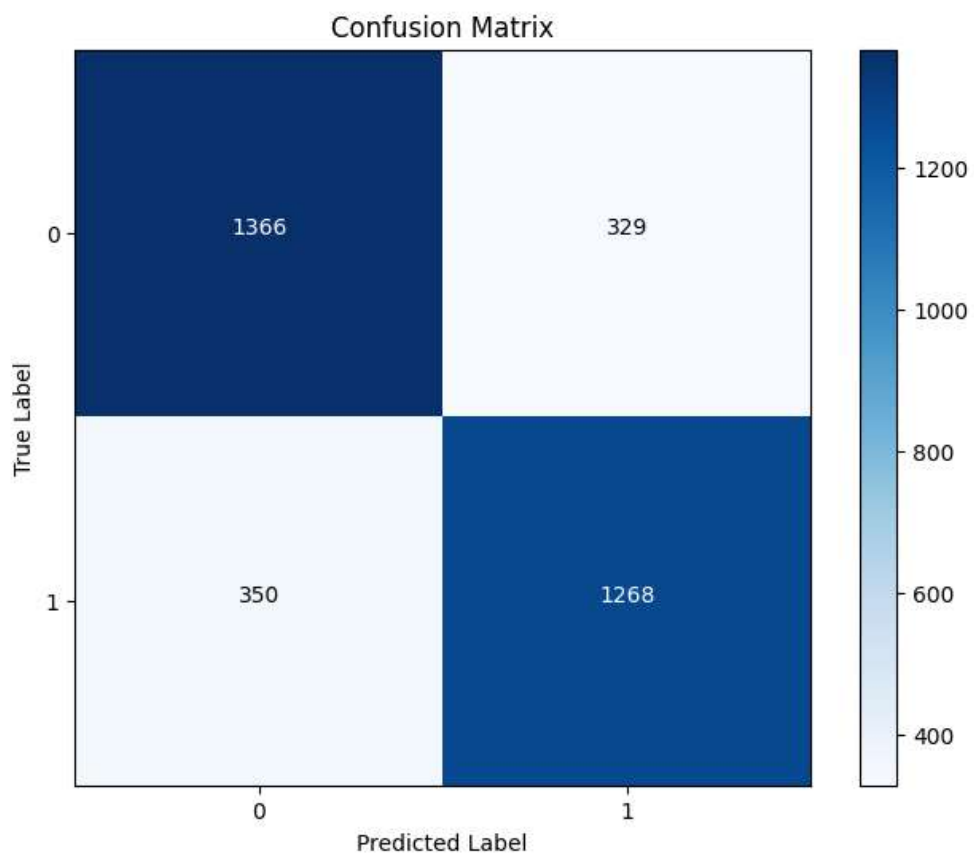


Рисунок 19 – Матрица ошибок SVM

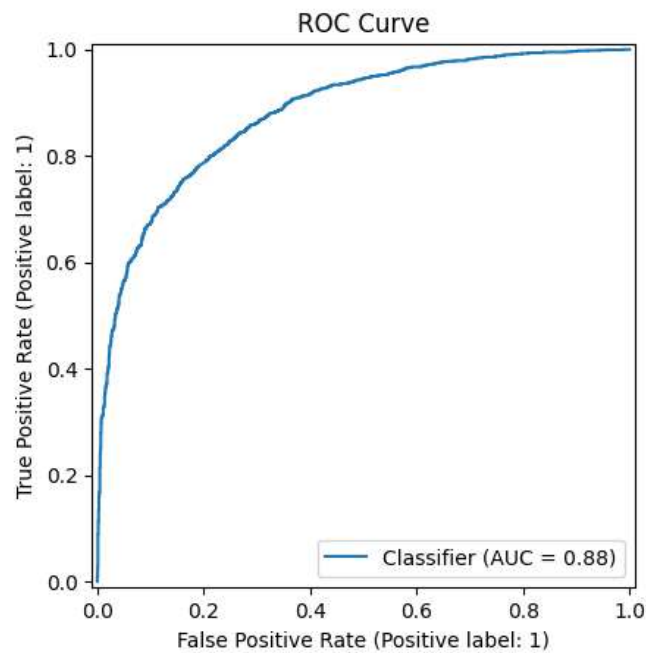


Рисунок 20 – ROC-кривая

Обучил модель Random Forest и предсказал значения на тестовой выборке (Рисунок 21).

```

from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier(n_estimators=100)
rf_model.fit(x_train, y_train)

RandomForestClassifier
RandomForestClassifier()

[79] y_pred = rf_model.predict(x_test)
     y_pred_proba = rf_model.predict_proba(x_test)[: , 1]

```

Рисунок 21 – Обучение RF

Вывел метрики качества модели (Рисунок 22). Построил матрицу ошибок (Рисунок 23). Построил ROC-кривые (Рисунок 24). Вывел диаграмму самых значимых признаков для случайного леса (Рисунок 25).

Accuracy: 0.78 ROC AUC Score: 0.88				
Classification Report:				
	precision	recall	f1-score	support
0.0	0.78	0.81	0.79	1695
1.0	0.79	0.75	0.77	1618
accuracy			0.78	3313
macro avg	0.78	0.78	0.78	3313
weighted avg	0.78	0.78	0.78	3313

Рисунок 22 – Метрики качества RF

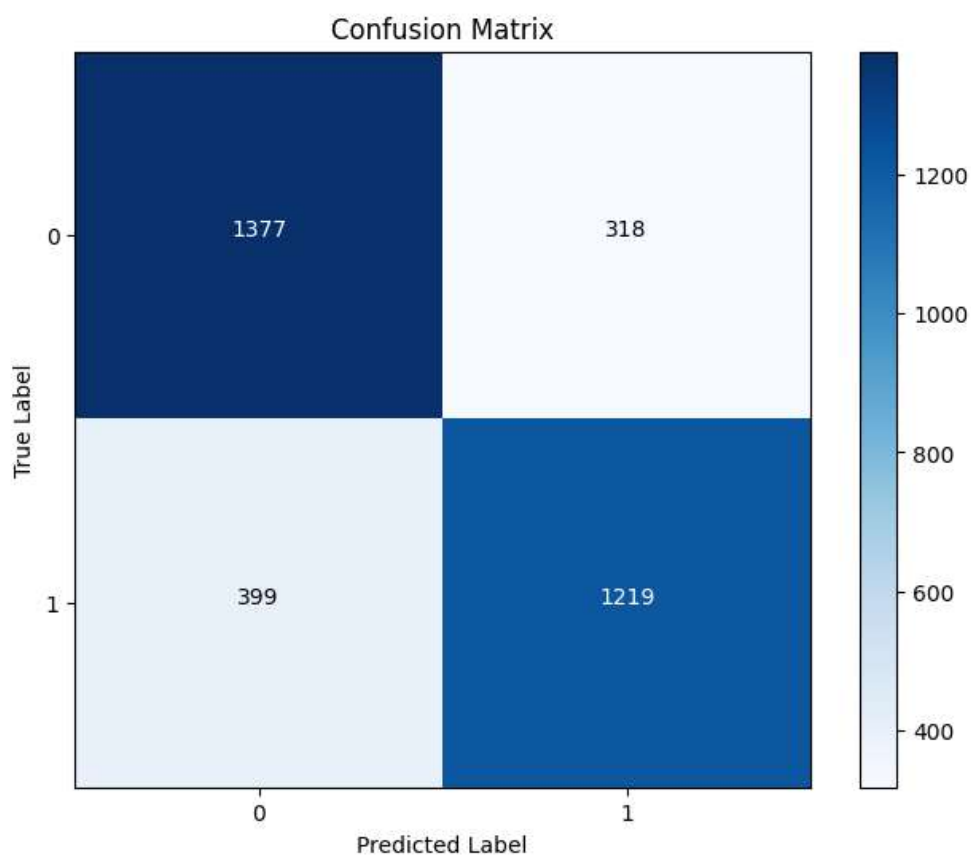


Рисунок 23 – Матрица ошибок RF

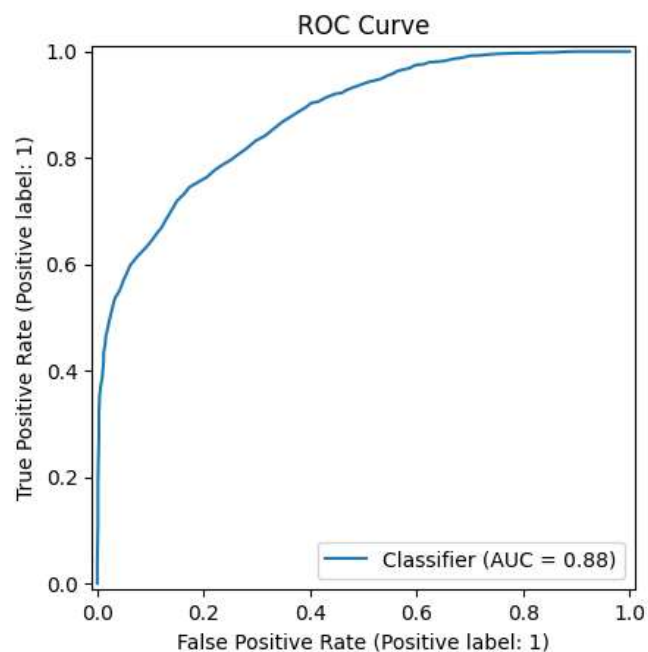


Рисунок 24 – ROC-кривая

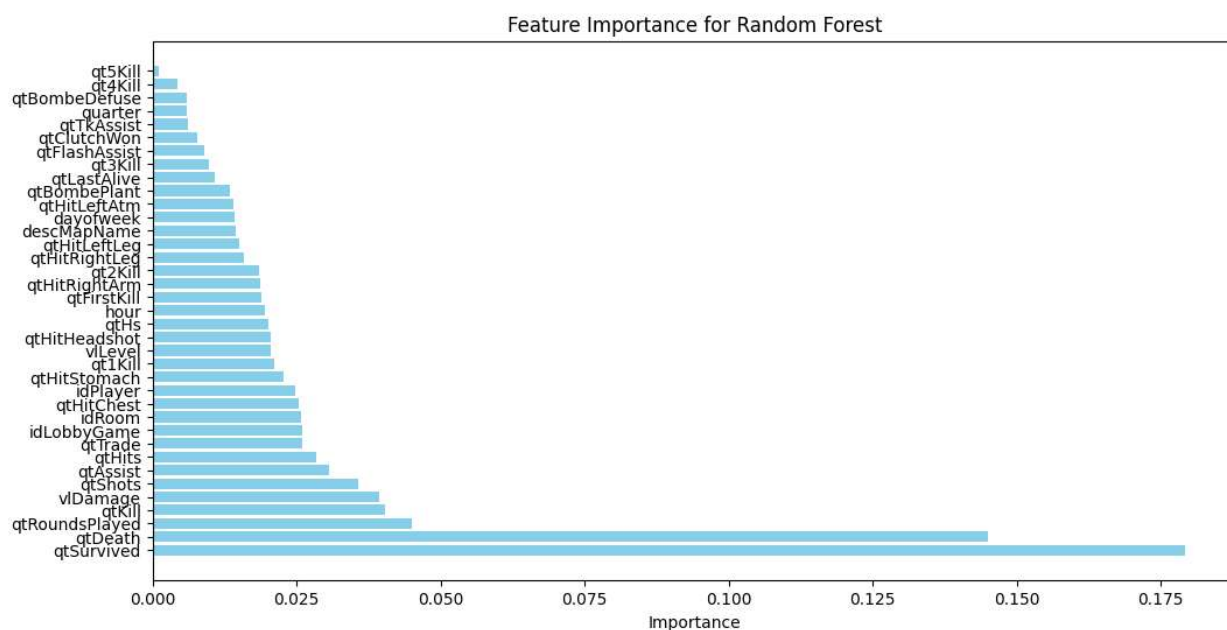


Рисунок 25 – Значимость параметров для случайного леса

По данной диаграмме можно наблюдать, что количество смертей и их отсутствия имеет наибольшее влияние на результат.

Выводы

В рамках данной лабораторной работы была проведена предварительная обработка данных. Этот процесс включал визуализацию значимых признаков, очистку данных от пропусков, дубликатов и аномалий, а также нормализацию, что способствовало повышению качества обучения моделей. Анализ корреляций позволил выявить избыточные признаки и избежать проблемы мультиколлинеарности.

На этапе построения моделей были использованы три популярных метода: К-ближайших соседей (KNN), Машина опорных векторов (SVM) и Случайный лес. Каждый из этих алгоритмов имеет свои особенности и требует тщательной настройки параметров для достижения оптимальных результатов.

Оценка качества моделей проводилась с использованием метрик precision, recall, F1-score и ROC-AUC, что позволило оценить их точность и полноту в решении задачи классификации.

Наихудшие результаты показал KNN, что предполагает его малую пригодность в данной ситуации. В целом точность получилось не большой, что связано в основном с контекстом данных.

Для улучшения метрик всех моделей, можно увеличить количество данных, провести тонкую настройку параметров для каждой из моделей. Однако это требует много времени и вычислительных ресурсов.

Приложения

Ссылка на блокнот в google.colab:

<https://colab.research.google.com/drive/1WVeNzEN1vzKgCR0IgPSt7s2Zl6Sl7V2V#scrollTo=bzjqBteAZPy>

В